IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Conocer los datos con pandas

Teoría, práctica y, al final, material de consulta

Sprint 6 · S06_01 · teoría breve, 55 min de práctica y material de consulta

PANDAS · LA TABLA

Una tabla con nombre: el DataFrame

pandas es la librería de Python para trabajar con tablas; una tabla de pandas es un **DataFrame**: cada **fila** es una reserva y cada **columna**, una característica. **NaN** marca un hueco: un dato que falta.

reserva	canal	noches	precio_noche	fecha_llegada	cancelada
101	web	3	85	2026-03-10	0
102	directo	2	120	2026-03-14	0
103	web	5	70	2026-04-02	1
104	agencia	4	NaN	2026-04-09	1
105	web	2	95	2026-05-21	1

Mirar la tabla antes de modelar se llama EDA: análisis exploratorio de datos.

LOS CUATRO PRIMEROS COMANDOS

Siempre se empieza así

shape

Cuántas **filas** y cuántas **columnas** tiene la tabla.

info

El **tipo** de cada columna y cuántos valores **no nulos** tiene.

head

Las **primeras filas**, para ver cómo es la tabla.

describe

Media, mínimo y máximo de las columnas numéricas.

Si los no nulos de info son menos que las filas, faltan valores.

Al leer el fichero, **parse_dates** hace que la fecha se lea **como fecha** y no como texto.

FILTRAR

Quedarse solo con lo que interesa

reserva	canal	¿canal es web?	¿Se queda?
101	web	Verdadero	**Sí**
102	directo	Falso	No
103	web	Verdadero	**Sí**
104	agencia	Falso	No
105	web	Verdadero	**Sí**

& significa «y»

Se cumplen **las dos** condiciones a la vez.

| significa «o»

Basta con que se cumpla **una**.

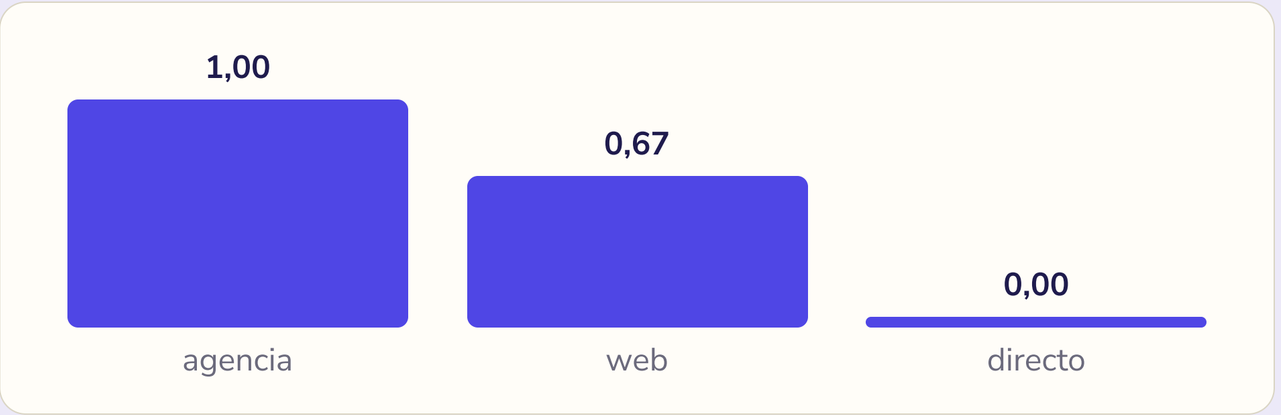
Entre paréntesis

Cada condición lleva el suyo. Con **and** u **or**, pandas da error.

AGRUPAR

La media de un 0/1 es una tasa

canal	cancelada
web	0
directo	0
web	1
agencia	1
web	1



Agrupar reparte las filas en grupos y calcula **una media por grupo**. Como **cancelada** vale 0 o 1, esa media es la **tasa de cancelación**.

- ✓ Compruébalo a mano: web = (0 + 1 + 1) / 3 = 0,67

UNIR TABLAS

Unir dos tablas con merge

Reservas

reserva	mes_llegada
101	3
102	3
103	4
104	5

Temporadas

mes_llegada	temporada
3	baja
4	alta
6	alta

Resultado

reserva	mes_llegada	temporada
101	3	baja
102	3	baja
103	4	alta
104	5	NaN

Se unen por la **columna común** (**mes_llegada**). Con **how="left"** se conservan **todas las reservas**, aunque no tengan pareja: la 104 queda con **NaN**.

Cuenta las filas antes y después: si cambian, la unión ha perdido o duplicado reservas.

ELEGIR EL GRÁFICO

El gráfico depende del dato

Quiero ver…	Gráfico	Cómo se lee
Una numérica	Histograma	Cuántas reservas caen en cada tramo
Una categórica	Barras	Una barra por categoría: cuántas hay
Numérica por categoría	Diagrama de cajas	Caja, mediana y puntos sueltos (ver abajo)
Dos numéricas	Dispersión	Un punto por reserva: ¿suben juntas?
Una fecha	Línea	El eje horizontal es el tiempo



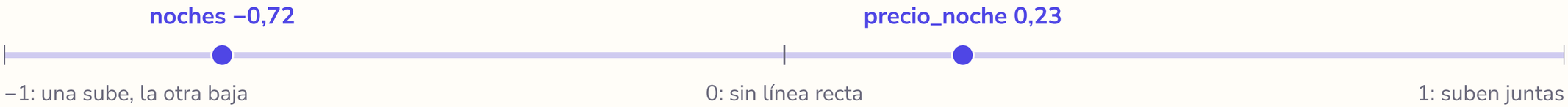
pandas dibuja con **matplotlib**; **seaborn** añade gráficos estadísticos con menos código.

CORRELACIONES

¿Qué variables se mueven juntas?

	noches	precio_noche	cancelada
noches	1,00	−0,62	−0,72
precio_noche	−0,62	1,00	0,23
cancelada	−0,72	0,23	1,00

La correlación va de **−1 a 1**: el **signo** dice la dirección y el **tamaño**, la fuerza. Solo detecta relaciones «en línea recta».



Correlación no es causalidad: que se muevan juntas no prueba que una cause la otra.

CALIDAD DEL DATO

Primer chequeo: duplicados y vacíos

reserva	canal	noches	precio_noche
101	web	3	85
102	directo	2	NaN
103	web	5	70
103	web	5	70
104	agencia	0	95
105	web	2	NaN

Duplicados

La reserva **103** aparece **dos veces**.

Vacíos

2 de 6 precios: un **33,3 %**.

Imposibles

0 noches en la 104: no puede ser.

Primero se **mide** el defecto; limpiarlo es el tema del siguiente cuaderno, S06_02.

MINI-RETO · CONTARLO AL CLIENTE

De la cifra a la frase del gerente

Poco útil	Útil para un gerente
La media de **cancelada** por tarifa es 0,75 y 0,25.	Con tarifa flexible **cancela el 75 %** de las reservas, **3 veces más** que con la no reembolsable (25 %).

Una cifra

Que hayas calculado tú.

Una comparación

Frente a otro grupo o periodo.

Sin jerga

Sin **groupby** ni **NaN**: se entiende sin saber Python.

Mini-reto: escribe en `hallazgos` tres frases así, con una cifra cada una.

PRÁCTICA · CUADERNO S06_01

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Sección 2	Cargar la tabla de reservas y mirar tipos y resumen	Clase
Ejercicio 1	Cargar la tabla en **df** y guardar el número de filas y de columnas	Clase
Ejercicio 2	Tasa de cancelación por depósito, de mayor a menor	Clase
Ejercicio 3	Unir con las temporadas y comparar la tasa de cada una	Clase
Ejercicio 4	Mediana de precio por tipo de habitación, con su gráfico	Casa
Ejercicio 5	La variable más ligada a la cancelación	Casa
Ejercicio 6	Filas duplicadas y porcentaje de nulos de **pais_origen**	Clase
Mini-reto	Tres hallazgos para el cliente, con una cifra cada uno	Casa

La «Solución de referencia» de cada ejercicio está plegada: se abre solo después de intentarlo. Tras esta diapositiva: **material de consulta** con el código y las aclaraciones de cada ejercicio.

CONSULTA

Material de consulta: código y aclaraciones

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta. Está en el **mismo orden que el cuaderno**: cada ejercicio trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Cargar y mirar la tabla	13 y 14
2	Filtrar y agrupar	15 a 18
3	Unir tablas con merge	19 y 20
4 (casa)	Gráfico de barras con la mediana	21 y 22
5 (casa)	Variable más ligada a la cancelación	23 y 24
6	Duplicados y vacíos	25 y 26
Mini-reto (casa)	Tres hallazgos	27
Ampliación (casa)	Datos grandes	28

CONSULTA · CARGAR Y MIRAR

Cargar la tabla y mirarla (ejercicio 1)

```
# CSV: fichero de texto con una reserva por línea y los valores separados por comas
df = pd.read_csv("mini.csv", parse_dates=["fecha_llegada"])
filas, columnas = df.shape    # (5, 6): 5 filas y 6 columnas
df.info()                     # precio_noche: 4 no nulos de 5
df.head()                     # las 5 primeras filas
df.describe()                 # noches: media 3.2, de 2 a 5
```

parse_dates

Lee la fecha **como fecha** (**datetime64**) y no como texto (**object**).

¿Faltan datos?

Si los **no nulos** de **info** son menos que las filas, **faltan valores**.

CONSULTA · ACLARACIONES · CARGAR Y MIRAR

Leer los cuatro comandos (ejercicio 1)

read_csv

Lee un fichero de texto con una reserva por línea. Con **parse_dates** esas columnas se leen **como fechas**.

shape da dos números

filas, columnas = df.shape los reparte en dos variables: con **(5, 6)**, **filas** vale 5 y **columnas** vale 6.

info y los no nulos

precio_noche tiene **4 no nulos** de 5 filas: falta uno. **object** es texto y **datetime64** es fecha.

describe resume números

Recuento, media, mínimo, cuartiles y máximo de las columnas **numéricas**: el texto no aparece.

CONSULTA · FILTRAR

Filtrar con condiciones (ejercicio 2)

Una **condición** da verdadero o falso para cada fila, y pandas se queda con las filas verdaderas.

```
df[df["canal"] == "web"]                                # 3 filas: 101, 103 y 105
df[(df["noches"] > 3) & (df["canal"] == "agencia")]     # 1 fila: 104
df[(df["canal"] == "directo") | (df["noches"] > 4)]     # 2 filas: 102 y 103
```

& significa «y»

Se cumplen **las dos** condiciones a la vez.

| significa «o»

Basta con que se cumpla **una**.

Entre paréntesis

Cada condición lleva el suyo. Con **and** u **or**, pandas da error.

CONSULTA · ACLARACIONES · FILTRAR

Leer un filtro (ejercicio 2)

Una condición es una serie

`df["canal"] == "web"` da **una respuesta por fila**: **True, False, True, False, True**.

df[condición]

Se queda con las filas donde la condición es **True**: la 101, la 103 y la 105.

Por qué los paréntesis

`&` y `|` se evalúan **antes** que `==` o `>`: sin paréntesis, pandas da error.

and y or no sirven

Python no sabe resolverlos con series enteras y avisa de que **el valor de verdad es ambiguo**.

CONSULTA · AGRUPAR

Agrupar y calcular la tasa (ejercicio 2)

canal	cancelada
web	0
directo	0
web	1
agencia	1
web	1

```
tasa = df.groupby("canal")["cancelada"].mean().round(2)
# agencia 1.00 · directo 0.00 · web 0.67
tasa.sort_values(ascending=False)
# agencia 1.00 · web 0.67 · directo 0.00
```

En el ejercicio 2 se cambia **"canal"** por **"deposito"** y se ordena de mayor a menor.

✓ Compruébalo a mano: web = (0 + 1 + 1) / 3 = 0,67

CONSULTA · ACLARACIONES · AGRUPAR

Leer groupby y mean (ejercicio 2)

groupby("canal")

Crea **un grupo por canal**: web (101, 103 y 105), directo (102) y agencia (104).

["cancelada"].mean()

Calcula la media de cancelada **dentro de cada grupo**: como vale 0 o 1, es la proporción de canceladas.

round(2)

Deja **dos decimales**: 0.6666... pasa a 0.67.

sort_values(ascending=False)

Ordena de **mayor a menor**. Sin ascending=False ordena de menor a mayor.

CONSULTA · UNIR TABLAS

Unir tablas con merge (ejercicio 3)

```
# on: la columna común · how="left": conserva todas las reservas
resultado = reservas.merge(temporadas, on="mes_llegada", how="left")
len(reservas), len(resultado)          # (4, 4): no se perdió ninguna
resultado["temporada"].isna().sum()    # 1: la 104 no tiene pareja
```

Con las tablas de ejemplo de la teoría: **reservas** (4 filas) y **temporadas** (3 filas). En el ejercicio 3, la tabla se llama **df** y el resultado, **df_temp**.

CONSULTA · ACLARACIONES · UNIR TABLAS

Leer merge (ejercicio 3)

on="mes_llegada"

La columna que **aparece en las dos tablas**: se usa para emparejar cada reserva con su temporada.

how="left"

Conserva **todas las filas de la tabla de la izquierda**; si no hay pareja, las columnas nuevas quedan en **NaN**.

Contar antes y después

len(reservas) y **len(resultado)** deben coincidir. Si salen **más** filas, la otra tabla repite alguna clave.

Una fila por mes

temporadas debe tener **un solo mes por fila**: dos filas para el mismo mes duplicarían las reservas.

CONSULTA · ELEGIR EL GRÁFICO

Barras con la mediana (ejercicio 4, casa)

```
# precios: estandar 60 y 70 · superior 90, 100 y 110 · familiar 140 y 150
mediana = df.groupby("tipo")["precio"].median()    # estandar 65 · familiar 145 · superior 100
mediana.plot(kind="bar")                           # una barra por tipo
mediana.idxmax()                                   # 'familiar'
```

En el ejercicio 4 se agrupa por **tipo_habitacion** y se ordena con **sort_values()** antes de dibujar. Salen cuatro barras, de menor a mayor, y **tipo_mas_caro** vale **suite**.

CONSULTA · ACLARACIONES · GRÁFICOS

Leer el gráfico de barras (ejercicio 4, casa)

median y no mean

La mediana es el valor del medio: **no la arrastran** unos pocos precios muy altos.

plot(kind="bar")

Una **barra por grupo**: en el eje horizontal van los tipos y en el vertical, la mediana.

idxmax()

Devuelve el **nombre** del grupo con el valor más alto (**familiar**), no el valor (145).

Cajas y dispersión

Para diagramas de cajas o de dispersión se usa **seaborn**, con menos código que matplotlib.

CONSULTA · CORRELACIONES

Calcular la variable más ligada (ejercicio 5, casa)

```
r = corr["cancelada"].drop("cancelada")  # noches -0.72 · precio_noche 0.23
r.abs()                                  # noches 0.72 · precio_noche 0.23
r.abs().idxmax()                         # 'noches': la más ligada, aunque sea negativa
```

corr es la matriz de correlaciones de la teoría. En el ejercicio 5, **var_mas_ligada** vale **"antelacion_dias"**, con una correlación de 0,28.

CONSULTA · ACLARACIONES · CORRELACIONES

Leer la variable más ligada (ejercicio 5, casa)

corr["cancelada"]

La columna de la matriz con las correlaciones de **cada variable con cancelada**.

drop("cancelada")

Quita la correlación de **cancelada consigo misma**, que siempre vale 1.

abs()

Quita el signo: **–0,72 y 0,72 cuentan igual**. Sin él, una correlación muy negativa quedaría la última.

idxmax()

Devuelve el **nombre** de la variable con el valor más alto.

CONSULTA · CALIDAD DEL DATO

Contar duplicados y vacíos (ejercicio 6)

reserva	canal	noches	precio_noche
101	web	3	85
102	directo	2	NaN
103	web	5	70
103	web	5	70
104	agencia	0	95
105	web	2	NaN

```
df.duplicated().sum()                    # 1
df["precio_noche"].isna().sum()          # 2
df["precio_noche"].isna().mean() * 100   # 33.33…
df["noches"].min()                       # 0
```

isna() marca con **True** cada celda vacía, y la **media** de **True**/**False** es la **proporción**: 2 de 6 = 33,3 %. Un **valor imposible** (0 noches) también es un defecto.

CONSULTA · ACLARACIONES · CALIDAD DEL DATO

Leer el chequeo (ejercicio 6)

duplicated()

Marca como repetida **la segunda copia** (la 103); **sum()** cuenta cuántas hay.

isna() y la media

True vale 1 y **False** vale 0: la **media es la proporción** de vacíos, 2 de 6 = 0,333.

* 100 y round

*** 100** pasa la proporción a **porcentaje**; **round(x, 1)** deja **un decimal**.

min() y los imposibles

noches.min() vale 0: una reserva de **0 noches** es un valor imposible.

CONSULTA · ACLARACIONES · MINI-RETO

Escribir los tres hallazgos (mini-reto, casa)

Una lista de tres textos

`hallazgos = ["...", "...", "..."]`: cada texto es **una frase para el gerente**.

Una cifra tuya

Calcúlala con lo aprendido: una tasa por canal, la mediana de un tipo, una diferencia entre temporadas…

Una comparación

«El 26 % con depósito total, **frente al** 54 % sin depósito»: la cifra sola no dice nada.

Qué significa

Termina con lo que importa para el cliente, **sin nombres de funciones** ni jerga.

CONSULTA · AMPLIACIÓN · DATOS GRANDES (CASA)

Cuando la tabla no cabe en memoria

Truco	Qué hace	Ejemplo
Leer solo lo necesario	Menos columnas y tipos pequeños desde la lectura	`pd.read_csv(ruta, usecols=["canal", "noches"])`
Leer por trozos	Procesa 100 000 filas cada vez, sin cargar todo	`for t in pd.read_csv(ruta, chunksize=100_000):`
Tipos más pequeños	`int8` en vez de `int64`; `category` para textos repetidos	`df["noches"].astype("int8")`
Formato columnar	Parquet: más compacto y rápido de leer que el CSV	`df.to_parquet("datos.parquet")` (necesita `pyarrow`)
Otras herramientas	Cuando pandas ya no basta	Polars, Dask o una base de datos

8 MB → 1 MB

Una columna de **1 000 000** de enteros pequeños ocupa **8 MB** como **int64** y **1 MB** como **int8**.